

INTRODUCTION TO PROGRAMMING LANGUAGES

What is a language?

- **A language is a structured system of communication.**
- **Language consists of vocabulary and grammar.**

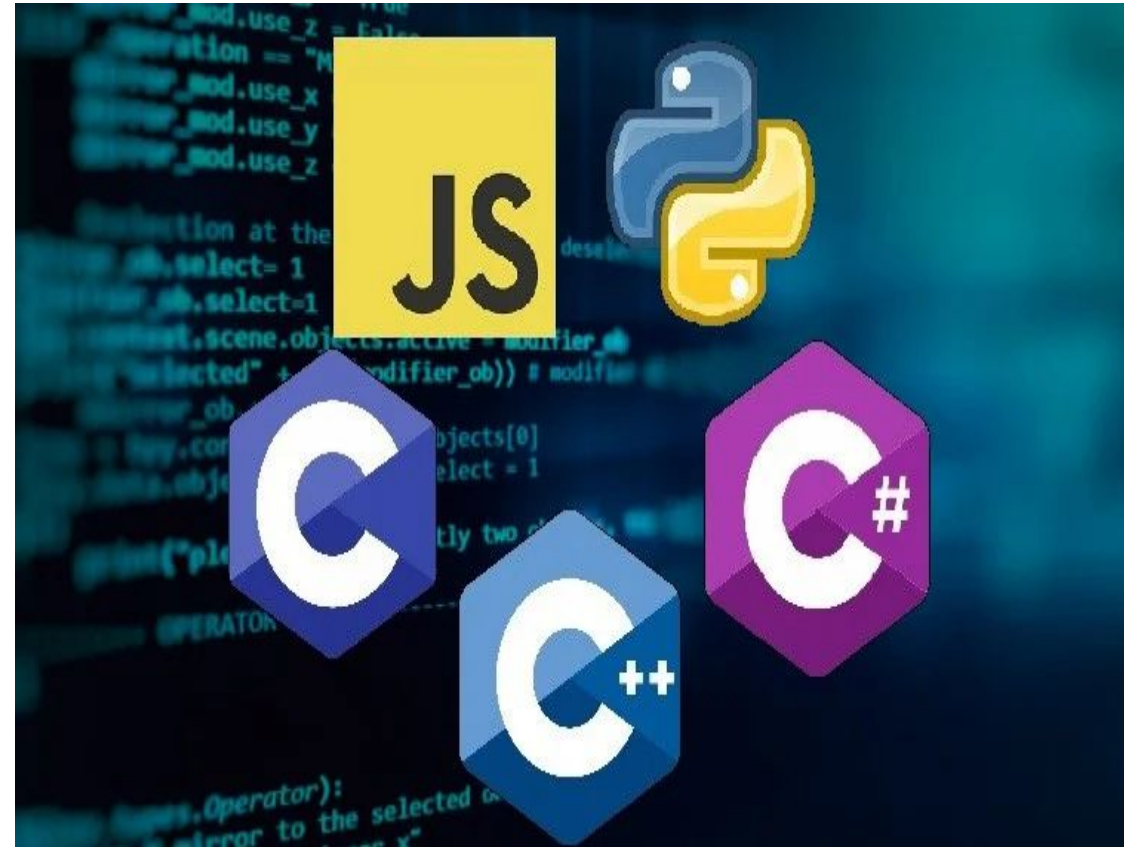
Example:

English, Hindi, Tamil, Malayalam



What is a programming language?

- **Programming languages are used to write instructions that computers can understand and execute.**
- **Each language has its own syntax (rules) and vocabulary.**
- **Programs are executed through compilation or interpretation.**

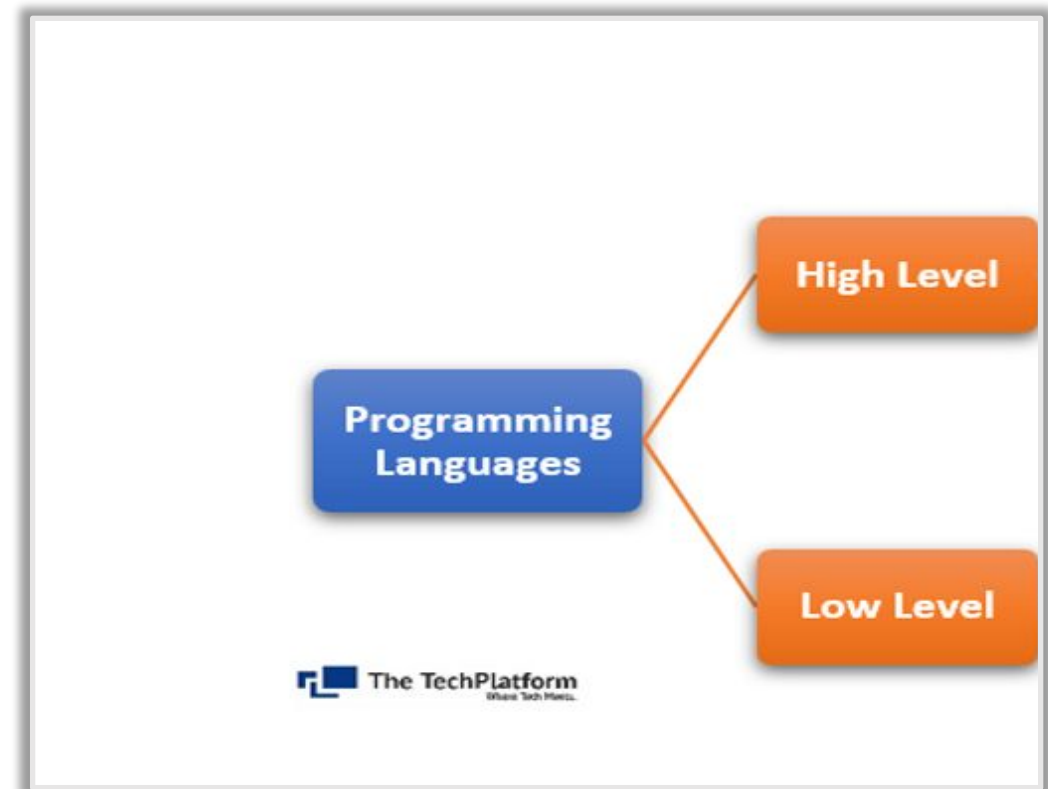


Types of Programming Languages

1. **High-level languages** -Easy to read, write and understand.
2. **Low-level languages** - closely tied to the computer's hardware, but are harder to use.

Examples

□ Python, Java, JavaScript, C , C++



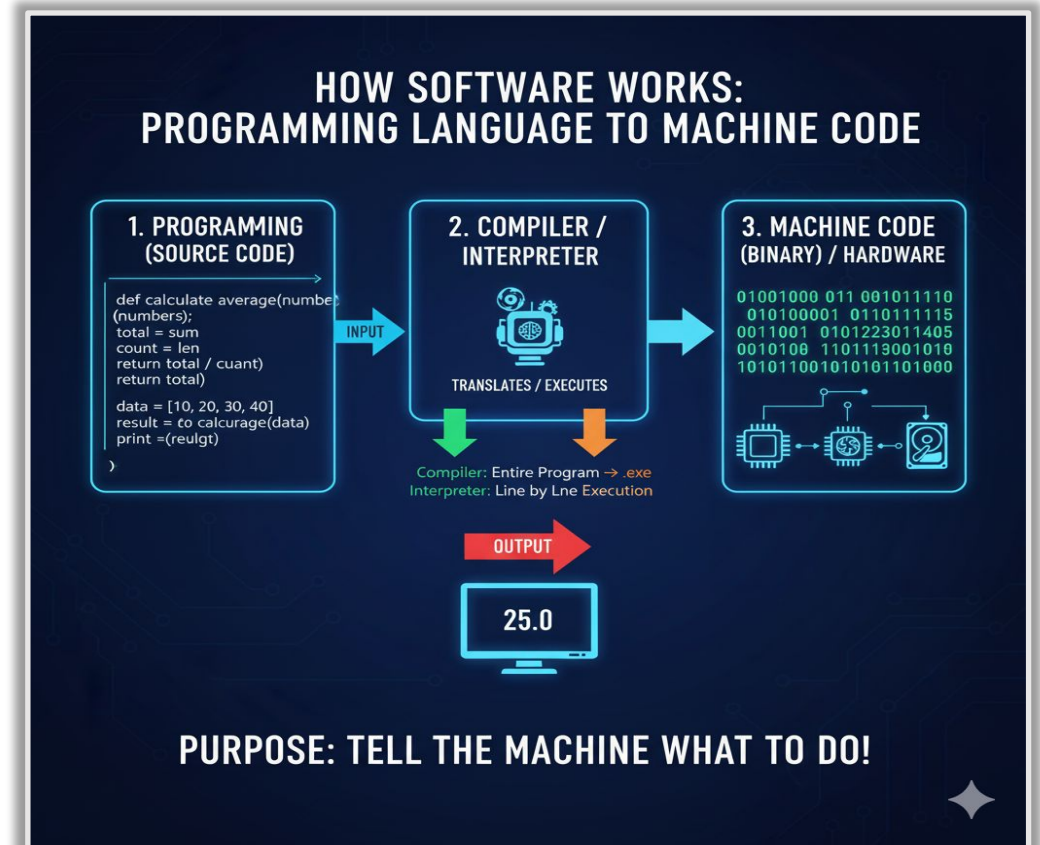
Translator of Programming Languages

1. Compiler

- ❑ translates a high-level programming language into machine code or bytecode before running the program.
- ❑ **Converts all code at once, creates executable file**
- ❑ Example : C, Java, C++

2. Interpreter

- ❑ **reads and executes code line-by-line**
- ❑ Example: Python, JavaScript, and Ruby



Do compilers or interpreters only translate high-level languages?

- ❑ **Compilers and interpreters are translators for high-level languages.**
- ❑ They take code written in **high-level languages** and **translate it** into a form the computer can understand — usually **machine code** or **intermediate code**.

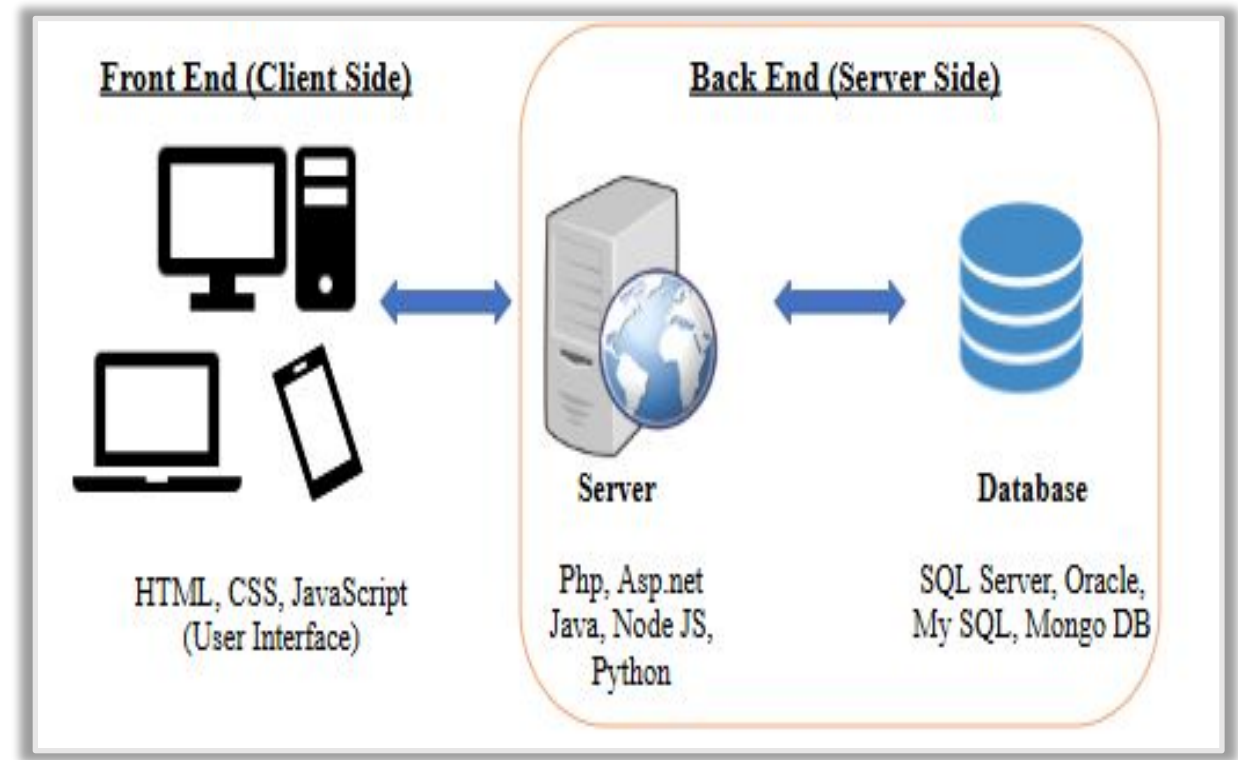
Feature	Compiler	Interpreter
Input	High-level language	High-level language
Output	Machine code (or intermediate code)	Executes line-by-line (no separate output file)
Execution speed	Fast (after compilation)	Slower (interprets line-by-line)
Examples	C, C++, Java (JVM), Go	Python, JavaScript, PHP, Ruby

INTRODUCTION TO PYTHON



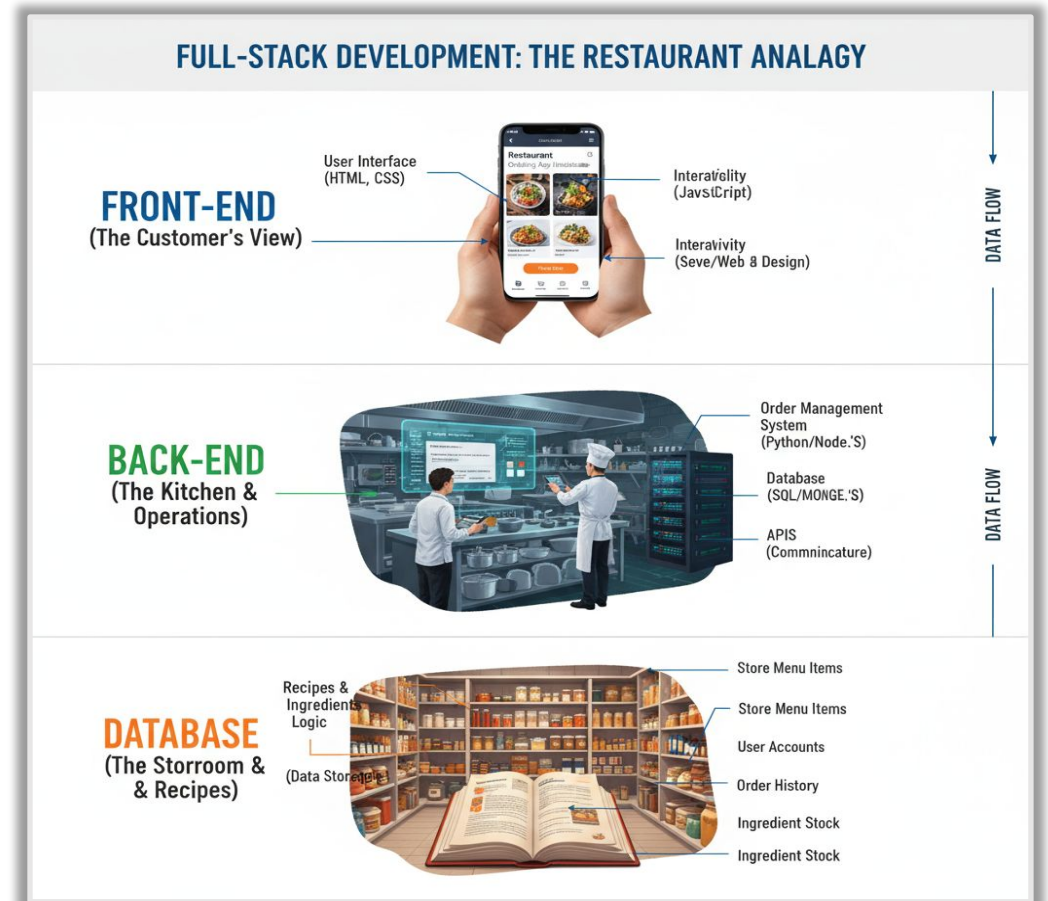
FULL STACK DEVELOPMENT

Full stack development refers to the practice of building both the front-end (client-side) and back-end (server-side) parts of a web application or software system.



What Full Stack Development Includes?

- ❑ **Front-End Development:**
Creating the user interface and experience using HTML, CSS, JavaScript, and frameworks like React, Angular, or Vue.js.
- ❑ **Back-End Development:**
Building server logic, databases, and APIs using languages such as Python, Java, PHP, or Node.js, and managing databases like MySQL, PostgreSQL, or MongoDB.
- ❑ **Database Management:**
Designing, implementing, and maintaining databases for efficient data storage and retrieval.



What is python?

- **General-purpose programming language**
- **Dynamically typed language**
- **High-level programming language**
- **Compiled and interpreted language**
- **Object-oriented programming language**



HISTORY OF PYTHON



- Created by **Guido van Rossum**, a Dutch programmer, in the late 1980s at Centrum Wiskunde & Informatica in Netherland.
- His goal was:
 1. **create an advanced scripting language based on ABC and Modula-3**
 2. **improve the readability of code**, allowing developers to **write codes in fewer lines.**
- The name "**Python**" was chosen by Van Rossum, a fan of the British comedy series "**Monty Python's Flying Circus**," to reflect a sense of fun and approachability in programming.
- First public version, **Python 0.9.0**, was released in **February 1991**.
- **Latest version – 3.14, released on Oct 7,2025**

Features of Python

- ❑ **Easy to learn and read with simple syntax**
- ❑ **Free and open source**
- ❑ **Cross-platform (runs on Windows, macOS, Linux)**
- ❑ **Large standard library with many built-in modules**
- ❑ **Dynamically typed (variable types decided at runtime)**
- ❑ **Interpreted language (executes code line-by-line)**
- ❑ **Supports multiple programming styles (procedural, OOP, functional)**
- ❑ **Strong community support with extensive resources**
- ❑ **Great for automation, web development, data science, and more**

Difference in number of lines of code

In C :

```
#include <stdio.h>
#include <conio.h>
void main()
{
    printf("Hello World");
    getch();
}
```

In Python:

```
print("Hello World")
```


WHY PYTHON?

- **Easy to Learn and Read**
- **Versatility**
- **Rich Libraries and Frameworks**
- **Strong Community Support**
- **Rapid Development**
- **Cross-Platform Compatibility**



Where is Python Used?



Job Opportunities



Major Websites Built with Python



ENVIRONMENTAL SETUP

- ❑ Python IDLE-Integrated Development And Learning Environment
- ❑ Visit the link <https://www.python.org/downloads/> to download the latest release of Python.
- ❑ Add Python to Path
- ❑ Visual Studio Code
- ❑ Visit the link <https://code.visualstudio.com/download> and download.



PYTHON IDENTIFIERS

An **identifier** is the **name** used to identify a **variable, function, class, module, or object** in Python.

Example: `name = "Sreeraj"` # 'name' is an identifier (variable)

Rules:

- ❑ Starts with a letter A to Z or a to z or an underscore (`_`) followed by zero or more letters, underscores and digits (0 to 9).
- ❑ No special characters `@`, `$`, and `%` within identifiers.
- ❑ Do not start with digits; use digits in between or end.
- ❑ Case sensitive programming language.
- ❑ Class names start with an uppercase letter.
- ❑ Keywords are reserved and can't be used as identifiers.

PYTHON KEYWORDS

Keywords are reserved words used for specific purposes

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

VARIABLES

- ❑ **A variable is a named container that stores data in a program.**
- ❑ **Syntax:** Variable_name = Value
- ❑ **Note:** Variable name used to refer memory location.
- ❑ **Example:** name = "Sreeraj"
age = 25
- ❑ Difference in variable declaration in C++ and Python
C++ : int age = 25
Python : age = 25

Data Types in Python

DATA TYPE

1. Numeric → int , Float
2. Text Type → str
3. Sequence Type → list, tuple, range
4. Mapping Type → dict
5. Set Type → set
6. Boolean → bool
7. None Type → None

Mutable Data types

- List
- Set
- Dictionary

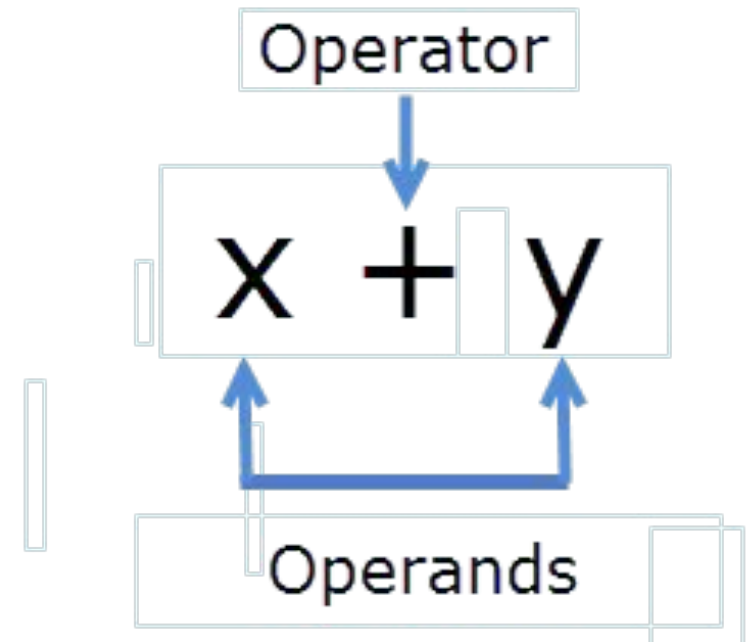
Immutable data type

Int
Float
Bool
Str
Tuple

Note: type() function which returns the type of the variable passed

Operators & Operand

Term	Meaning
Operator	Symbol that performs an action
Operand	Data/value on which the action is applied



Operators in Python

To perform a specific operation on different values and variables.

- Arithmetic Operators
- Comparison Operators
- Assignment Operators
- Logical Operators
- Membership Operators
- Identity Operators

Arithmetic Operators

Operator	Description
+ (Addition)	It is used to add two operands. For example, if $a = 20$, $b = 10 \Rightarrow a + b = 30$
- (Subtraction)	It is used to subtract the second operand from the first operand. If the first operand is less than the second operand, the value result negative. For example, if $a = 20$, $b = 10 \Rightarrow a - b = 10$
/ (divide)	It returns the quotient after dividing the first operand by the second operand. For example, if $a = 20$, $b = 10 \Rightarrow a / b = 2$
* (Multiplication)	It is used to multiply one operand with the other. For example, if $a = 20$, $b = 10 \Rightarrow a * b = 200$
% (reminder)	It returns the reminder after dividing the first operand by the second operand. For example, if $a = 20$, $b = 10 \Rightarrow a \% b = 0$
** (Exponent)	It is an exponent operator represented as it calculates the first operand power to second operand.
// (Floor division)	It gives the floor value of the quotient produced by dividing the two operands.

Comparison Operators

Operator	Description
==	If the value of two operands is equal, then the condition becomes true.
!=	If the value of two operands is not equal then the condition becomes true.
<=	If the first operand is less than or equal to the second operand, then the condition becomes true.
>=	If the first operand is greater than or equal to the second operand, then the condition becomes true.
>	If the first operand is greater than the second operand, then the condition becomes true.
<	If the first operand is less than the second operand, then the condition becomes true.

Assignment Operators

Operator	Description
=	It assigns the the value of the right expression to the left operand.
+=	It increases the value of the left operand by the value of the right operand and assign the modified value back to left operand. For example, if $a = 10$, $b = 20 \Rightarrow a + = b$ will be equal to $a = a + b$ and therefore, $a = 30$.
-=	It decreases the value of the left operand by the value of the right operand and assign the modified value back to left operand. For example, if $a = 20$, $b = 10 \Rightarrow a - = b$ will be equal to $a = a - b$ and therefore, $a = 10$.
*=	It multiplies the value of the left operand by the value of the right operand and assign the modified value back to left operand. For example, if $a = 10$, $b = 20 \Rightarrow a * = b$ will be equal to $a = a * b$ and therefore, $a = 200$.
%=	It divides the value of the left operand by the value of the right operand and assign the remainder back to left operand. For example, if $a = 20$, $b = 10 \Rightarrow a \% = b$ will be equal to $a = a \% b$ and therefore, $a = 0$.
**=	$a ** = b$ will be equal to $a = a ** b$, for example, if $a = 4$, $b = 2$, $a ** = b$ will assign $4 ** 2 = 16$ to a .
//=	$a // = b$ will be equal to $a = a // b$, for example, if $a = 4$, $b = 3$, $a // = b$ will assign $4 // 3 = 1$ to a .

Membership Operators

Operator	Description
in	It is evaluated to be true if the first operand is found in the second operand (list, tuple, or dictionary).
not in	It is evaluated to be true if the first operand is not found in the second operand (list, tuple, or dictionary).

Identity Operators

Operator	Description
is	It is evaluated to be true if the reference present at both sides point to the same object.
is not	It is evaluated to be true if the reference present at both side do not point to the same object.

Conditional Statements In Python

Used to perform different actions based on different conditions.

Types:

- if statement
- if-else statement
- Nested if statement
- if-elif-else statement

Conditional Statements In Python

1. **if Statement** : Executes a block of code if a specified condition is true.

Syntax:

```
if condition:
```

```
    # code to execute if condition is true
```

Example:

Question : Check eligibility for voting

```
age = 20
```

```
if age >= 18:
```

```
    print(" You are eligible")
```

Conditional Statements In Python

2. if –else Statement : Executes one block of code if the condition is true, and another block if the condition is false.

Syntax:

if condition:

code if true

else:

code if false

Example:

Question : Check eligibility for voting

```
age = 20
```

```
if age >= 18:
```

```
    print("You are an adult.")
```

```
else:
```

```
    print("You are a minor.")
```


Conditional Statements In Python

3. if-elif-else statement : Used to test multiple conditions. It executes the block of the first true condition.

Syntax:

if condition1:

 # code if condition1 is true

elif condition2:

 # code if condition2 is true

else:

 # code if all conditions are false

Example:

checks a number and prints whether it is positive, negative, or zero:

```
number = 10
```

```
if number > 0:
```

```
    print("The number is positive.")
```

```
elif number < 0:
```

```
    print("The number is negative.")
```

```
else:
```

```
    print("The number is zero.")
```

Conditional Statements In Python

4. Nested if statements: An if statement inside another if statement for checking multiple layers of conditions.

Syntax:

```
if condition1:
    if condition2:
        # code if both conditions are
true
    else:
        # code to be executed
```

Example:

checks a number is positive and even:

```
num = 10
if num > 0:
    if num % 2 == 0:
        print("The number is positive and even.")
    else:
        print("The number is positive but odd.")
else:
    print("The number is not positive.")
```

Conditional Statements In Python

5. Ternary conditional operator: A compact one-line if-else expression

Syntax:

```
x = value_if_true if condition else  
value_if_false
```

Example:

checks a person adult or minor:

```
age = 18  
status = "Adult" if age >= 18 else "Minor"  
print(status)
```

Loops In Python

- used to repeat a block of code multiple times, which helps avoid writing repetitive code.
- It provides code re-usability.
- Using loops, we do not need to write the same code again and again.
- Using loops, we can traverse over the elements of data structures (array or linked lists).
- There are mainly two types of loops in Python:
 - 1. For Loop**
 - 2. While Loop**

While Loop

- Repeats the code block as long as the given condition is true.
- else block- is executed when the condition given in the while statement becomes false. Like for loop, if the while loop is broken using break statement, then the else block will not be executed and the statement present after else block will be executed.

- **Syntax:**

while expression:

Statement(s)

Example: `i = 0`

`while i < 5:`

`print(i)`

`i += 1`

For Loop

- ❑ Iterates over a sequence (like lists, strings, tuples) or range of numbers.
- ❑ else statement with the for loop can be executed only when all the iterations are exhausted. If the loop contains any of the break statement, then the else statement will not be executed.

- ❑ **Syntax**

```
for var in iterable:
```

```
    # statements
```

- ❑ **range() Function**

- ❑ The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.
- ❑ Example: `fruits = ['apple', 'banana', 'cherry']`

```
    for fruit in fruits:
```

```
        print(fruit)
```

Nested Loop

- A loop inside another loop, used for multi-level iterations.
- Example:

```
for i in range(1, 4):  
    for j in range(1, 3):  
        print(i, j)
```

Loops

- ❑ **Pass** statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.
- ❑ **break** statement in Python **terminates the current loop** and resumes execution at the next statement, just like the traditional break found in C.
- ❑ **Continue** statement rejects all the remaining statements in the current iteration of the loop and moves the control back to the top of the loop.